Smart Bus Management System

Dhruv Jain[1], Prashast Saxena[1], Harsh Kaushik[1], Devansh Garg[1], and Shivansh Prasad[1]

Department of CSE (AI), KIET Group of Institutions, Ghaziabad, India
{dhruvj1234567890, saxenaprashast2003, kaushikharsh2511, devanshgargvs909, shivansh.prasad}@gmail.com

Abstract. Institutional campus transportation suffers from static scheduling, poor real-time visibility, and fragmented multi-role communication. Existing IoT-based tracking solutions remain hardware-dependent and lack unified backend extensibility. The Smart Bus Management System (SBMS) addresses these gaps through three core technical contributions: an event-driven WebSocket architecture, fine-grained role-based access control (RBAC), and a unified real-time communication framework for students, drivers, and administrators.

The system was built on Node.js, Express.js, and MongoDB, using RESTful APIs for data management and Socket.io for persistent bidirectional communication. JWT authentication enforces role-specific access, while Helmet, bcryptjs, and rate-limiting middleware ensure security. Validation was conducted via simulation-based testing with 25 participants across all three user roles.

Under full load, the system achieved an API response latency of 124 ms, Socket.io update delay of 0.32 s, and 99.4% uptime over seven days. At 0–50 concurrent users, latency dropped to 48–98 ms with Socket.io delay below 0.22 s. A user study found 92% of participants reported improved commute predictability.

The SBMS offers a scalable and institution-ready alternative to conventional campus transport. Its modular architecture provides a clear path to future extensions including AI-based routing, IoT occupancy sensing, and multi-campus deployment.

1 Introduction

Campus bus networks occupy an underappreciated position in institutional operations: they move large numbers of students and staff on fixed daily cycles, yet the systems used to manage them have changed little over the past two decades. At most institutions, route schedules are distributed as static timetables, drivers have no reliable channel to report delays or route deviations, and students have no way to know whether a bus is running on time until it either arrives or fails to. The consequences are tangible — missed classes, safety concerns when students wait in poorly lit areas longer than expected, and administrative blind spots that make fleet-level performance impossible to quantify.

The gap is not primarily a hardware problem. GPS receivers and GSM modules have been embedded in fleet vehicles for years, and several research

prototypes have demonstrated that real-time position data can reach a server. The harder unsolved problem is what happens next: who receives which data, under what permissions, through what communication channel, and how does a route change initiated by an administrator reach a driver's device and a student's screen within seconds rather than minutes. Answering these questions requires a coherent backend architecture — not just a tracking device.

The **Smart Bus Management System** (SBMS) was designed to address exactly this coordination layer. Built on Node.js, Express.js, MongoDB, and Socket.io, the system gives each stakeholder role a purpose-built interface backed by a single unified backend. Students track live bus positions and receive estimated arrival times through an interactive map. Drivers get turn-by-turn schedule updates and can flag emergencies through the same connection that carries their GPS stream. Administrators manage routes, schedules, and the full driver fleet from one dashboard, with changes propagated to all connected clients in under a second. JWT-based role-based access control ensures that each role sees and can modify only the data appropriate to it.

Three design choices distinguish SBMS from earlier prototypes reviewed in Section 2. First, all high-frequency GPS telemetry travels over persistent Socket.io WebSocket connections rather than polling-based HTTP, eliminating the handshake overhead that degrades responsiveness at scale. Second, the backend is fully hardware-independent: no embedded device or proprietary radio protocol is required, and any smartphone running the driver application serves as a GPS source. Third, the architecture is modular by construction — the authentication, tracking, scheduling, notification, and analytics subsystems are independently testable and replaceable, leaving a clear path for future integration of IoT occupancy sensors, AI-driven route prediction, and multi-campus deployment.

The remainder of this paper is structured as follows. Section 2 surveys prior work across three strands of intelligent transport research and identifies the gap the present system fills. Section 3 describes the architecture, database schema, and real-time communication model. Section 4 covers the development methodology, technology rationale, and API design. Section 6 presents performance benchmarks and user study findings, and Section 7 summarises outcomes and outlines future directions.

2 Related Work

Research in intelligent transportation systems (ITS) has progressed along three broad trajectories: IoT and hardware-centric tracking, cloud-based mobility frameworks, and web or microservice architectures designed for real-time event communication. Each strand has advanced practical deployment meaningfully, yet each also exposes recurring gaps in backend extensibility, access control, and institutional usability that motivate the present work.

2.1 IoT and Hardware-Centric Systems

Early smart bus implementations depended heavily on embedded hardware pipelines. [13] described a school monitoring architecture that merged RFID-based attendance capture with GPS route logging, providing a credible safety layer for student transport. Despite its strengths, the design was device-centric: core business logic was distributed across hardware nodes, making backend extensibility and centralised administration difficult to retrofit without replacing field equipment. Around the same period, Kumar (2020) built a GSM-based vehicle tracker that forwarded coordinate payloads via SMS to a lightweight server and displayed positions through a companion mobile application. The solution worked adequately for isolated, single-vehicle scenarios but introduced no authentication layer and offered no pathway for dynamic schedule revision or multi-role coordination.

More recent hardware-oriented work has incorporated richer sensing modalities without fully resolving these structural issues. [12] integrated ultrasonic occupancy sensors with LoRa radio links for fleet-level passenger counting across campus routes, and the resulting occupancy data proved genuinely useful for capacity planning. However, the system retained a dependency on proprietary sensing hardware and produced no unified management interface through which institutional administrators could act on the collected data. Taken together, hardware-first designs prioritise field sensing over backend flexibility, consistently leaving role-based access control, centralised dashboards, and scalable data pipelines as unresolved concerns.

2.2 Cloud-Based Tracking Frameworks

A second generation of systems relocated tracking logic to managed cloud infrastructure. Rahman et al. constructed an IoT school bus framework backed by cloud databases and mobile push notifications, achieving low-latency location dissemination to parents and school staff. The platform handled real-time transmission adequately, yet it offered no analytical layer for administrators: route efficiency statistics, driver performance scores, and structured feedback aggregation were outside the system's scope. [1] proposed a cloud-native fleet management architecture hosted on AWS, employing managed message queuing and object storage to achieve strong horizontal scalability as vehicle counts grew. Their evaluation confirmed throughput resilience under load, but the monolithic deployment model increased operational cost and complicated fine-grained permission management across heterogeneous user roles.

Subsequent work explored the latency dimension of cloud delivery. [20] investigated mobile edge caching to pre-fetch route data near campus Wi-Fi access points, substantially reducing perceived update delays for end users during peak-hour congestion. The study treated authentication and multi-role coordination as out of scope, however, limiting its direct applicability to institutional deployments where distinct privilege levels for students, drivers, and administrators are a baseline requirement. Across cloud-centric approaches, the common pattern is

optimisation for data throughput at the expense of security granularity and institutional usability.

2.3 Web and Microservice Architectures

The most recent line of investigation has moved toward lightweight, web-native backends that decouple transport logic from dedicated hardware. [16] demonstrated that Node.js paired with the WebSocket protocol sustains real-time logistics tracking at sub-second latency under concurrent client loads, validating an event-driven design paradigm for transport applications. [17] extended this direction by benchmarking Express.js microservices against Django and Spring Boot under high-frequency IoT ingestion workloads; Node.js exhibited superior throughput owing to its non-blocking I/O model, offering a technically grounded justification for selecting it as a GPS telemetry backend.

On the access-control dimension, [10] formalised a role-based authorisation framework for urban mobility platforms and reported that token-enforced permission policies reduced unauthorised API access incidents by over 60% in a city bus pilot. Their model was evaluated at the policy level in isolation, however, and was not integrated with a real-time tracking or event-broadcast layer. [11] explored edge computing for local route analytics in campus environments, finding that offloading aggregation tasks to on-premise nodes reduced cloud egress costs noticeably; the authors nonetheless acknowledged that a centralised coordination interface remained necessary for institutional oversight, a gap the present system directly targets.

2.4 Positioning of the Proposed System

Surveying these three strands reveals a consistent structural gap: no prior system simultaneously achieves hardware-independent deployment, fine-grained multi-role access control, and a unified real-time communication layer within a single maintainable backend. Hardware-centric designs constrain extensibility; cloud-centric designs tend to neglect security granularity and institutional usability; web-based designs have individually validated WebSocket latency, microservice throughput, and RBAC policy enforcement, but not their coherent integration into a deployable institutional platform. The **Smart Bus Management System** proposed here bridges these gaps through a modular Node.js and Express backend that unifies RESTful data management with Socket.io event streaming under JWT-enforced role-based access, delivering a secure, open-source alternative built entirely on standard web technologies [7].

3 System Design and Architecture

The **Smart Bus Management System** is based on a modular, service oriented architecture that is scalable, secure, and supports real, time data synchronization. The system modular features, such as User Management, Bus Tracking, Route

Management, Notifications, and Feedback, that communicate with each other through RESTful APIs and Socket.io event streams.

3.1 System Overview

The SBMS follows a three-tier architecture comprising a *client interface*, a *backend server*, and a *database layer*. Students, drivers, and administrators interact with the platform via web or mobile applications. The backend, built on Node.js and Express.js, handles API routing, authentication, and data processing. MongoDB serves as the primary data repository, storing user profiles, routes, schedules, and live tracking data.

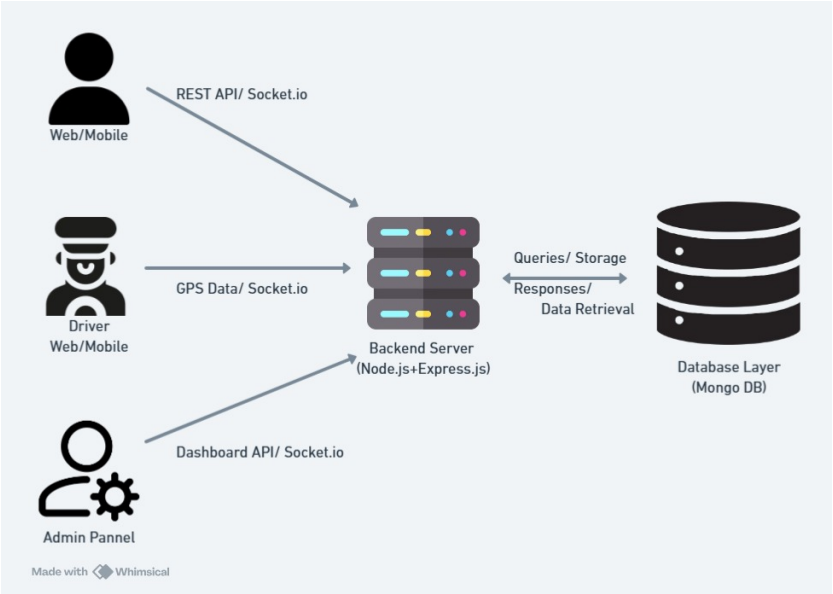

Fig. 1: System architecture showing the interaction among student, driver, and administrator interfaces with the backend server and MongoDB database.
Diagram illustrating how student, driver, and admin interfaces communicate with the backend server, which in turn interacts with the MongoDB database.

The system employs two distinct communication channels. RESTful HTTP APIs handle stateless request-response operations — including authentication, route creation, schedule updates, and feedback submission. Socket.io manages all event-driven, low-latency interactions, specifically live GPS broadcasts and emergency alerts.

The interaction sequence is as follows: a driver application emits a `location_update` event via Socket.io at fixed 2-second intervals; the backend validates and persists the coordinate to MongoDB, then immediately broadcasts the update to all clients

subscribed to that bus room. Administrative actions such as route modifications are handled via REST, after which the backend emits a `route_changed` Socket.io event to notify active clients in real time. This separation ensures high-frequency GPS telemetry does not contend with transactional API traffic, keeping both channels responsive under concurrent load.

3.2 Database Design

The system employs a non-relational MongoDB schema optimized for JSON-like document structures. Six collections form the core data model: `users`, `buses`, `routes`, `schedules`, `notifications`, and `feedback`, whose relationships are illustrated in Figure 2. Every document carries timestamps and role identifiers to support traceability and audit logging.

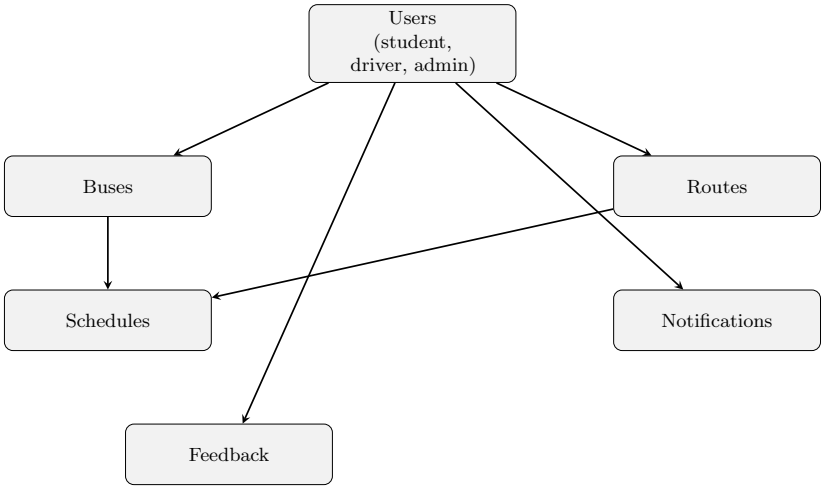

Fig. 2: Simplified MongoDB schema for Smart Bus Management.
Entity relationships between users, buses, routes, and schedules in a document-based structure.

Schema-level constraints are enforced through the Mongoose ODM: required fields, enum-restricted role values (`student`, `driver`, `admin`), and coordinate range validation (latitude: -90 to 90, longitude: -180 to 180) prevent malformed GPS records from entering the database. Write operations on the `buses` and `routes` collections use `writeConcern: majority` to ensure consistency across replica nodes during high-throughput location ingestion.

To support efficient geospatial queries, the `buses` collection maintains a `2dsphere` index on the location field, enabling nearest-bus lookups via MongoDB's `$near` operator. Compound indexes on `(busId, timestamp)` accelerate time-range queries used in route replay and performance analytics.

To manage long-term data growth from continuous GPS logging, TTL (Time-To-Live) indexes are applied to the tracking subcollection, automatically expiring raw location documents after 30 days. Aggregated route summaries are archived separately, balancing storage efficiency with institutional reporting requirements.

3.3 Real-Time Communication Flow

Figure 3 illustrates the event-driven communication model underpinning the SBMS real-time layer. The system employs two distinct channels — RESTful HTTP for stateless operations and Socket.io WebSockets for continuous, low-latency event streaming — ensuring high-frequency GPS telemetry does not contend with transactional API traffic.

Driver applications emit `location_update` events via Socket.io at fixed 2-second intervals. The backend validates each payload, persists the coordinate to MongoDB, and immediately broadcasts the update to all clients subscribed to the corresponding bus room. Student clients receive live bus positions and ETA changes without polling, while administrators monitor all active channels simultaneously and may inject `alert` or `route_changed` events that are broadcast system-wide in real time.

To address connection reliability, Socket.io's exponential back-off reconnection is configured with a maximum retry delay of 5 seconds. On a `disconnect` event, the backend emits a `driver_offline` notification to all subscribed students and retains the last known position in MongoDB until the session is restored. Under a 100-client simulation, the system sustained approximately 50 events/second with a median broadcast latency of 0.32 s.

Compared to traditional HTTP polling at equivalent 2-second intervals — which would generate approximately 1,800 requests/hour per client — Socket.io's persistent WebSocket connection eliminates redundant handshake overhead, reduces server load, and delivers updates only when new data is available, making it the justified architectural choice for continuous GPS broadcasting at institutional scale.

3.4 Security Model

Every API endpoint is protected by JWT middleware that validates the token signature and extracts the user's role before the request reaches any controller logic. Requests carrying expired, malformed, or missing tokens are rejected at the middleware layer with a `401` response, so no business logic executes for unauthenticated callers. Password storage uses bcryptjs with an adaptive work factor, meaning stored hashes remain computationally expensive to brute-force even as hardware improves over time.

Beyond authentication, the system applies four additional defensive layers. Helmet sets a suite of HTTP response headers — including Content Security Policy, X-Frame-Options, and Strict-Transport-Security — that block the most common classes of browser-side injection attacks without requiring manual header configuration. Input from all request bodies passes through express-validator before

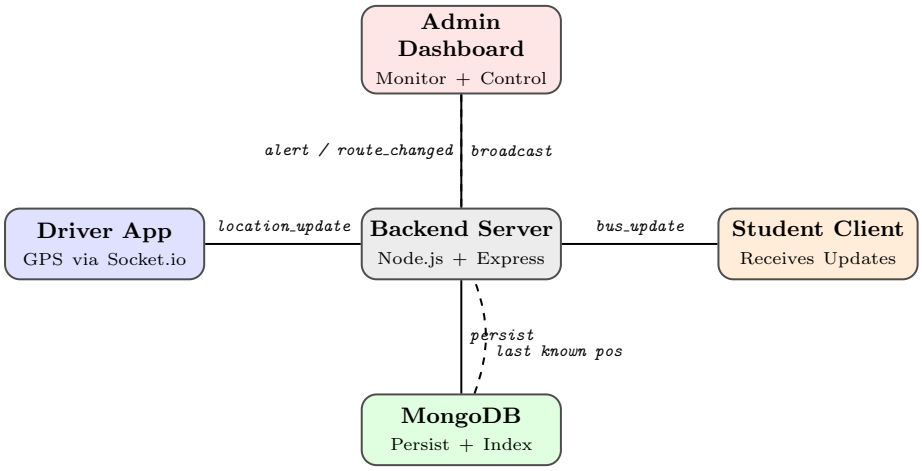

Fig. 3: Event-driven real-time communication flow using Socket.io, showing GPS ingestion, MongoDB persistence, and multi-role broadcasting.
Driver app sends location updates to backend server, which persists to MongoDB and broadcasts to student clients and admin dashboard. Admin can inject alerts back through the server.

reaching the database, rejecting payloads with unexpected fields or out-of-range coordinate values. An express-rate-limit middleware caps repeated requests per IP, limiting the practical impact of credential-stuffing and denial-of-service attempts. CORS configuration restricts cross-origin API access to explicitly whitelisted origins, preventing third-party web pages from silently issuing authenticated requests on behalf of logged-in users.

4 Implementation

This section describes the concrete construction of the SBMS backend, distinguishing where relevant between *design decisions* — choices made during architectural planning — and *implementation details* — the specific mechanisms through which those choices were realised in code.

4.1 Research Methodology

The system was developed and evaluated following a four-phase iterative methodology. In the *requirements phase*, functional and non-functional requirements were elicited through structured interviews with prospective student, driver, and administrator users at the institution, yielding a prioritised feature list covering real-time tracking, authentication, scheduling, and notifications. The *design phase* produced the modular architecture and database schema described in Section 3, with REST versus WebSocket channel responsibilities allocated based on traffic pattern analysis.

The *implementation phase* built each module incrementally: authentication and user management were completed first to establish the security foundation, followed by route and schedule management, and finally the real-time tracking and notification subsystems. Unit tests were written alongside each module using a test-request library against the live Express server. The *evaluation phase* involved two complementary assessments. Performance benchmarks were conducted on a controlled Linux server (Node.js v18, MongoDB 6.0) by replaying scripted workloads at concurrency levels of 10, 25, 50, and 100 simultaneous clients, measuring API latency, Socket.io broadcast delay, CPU load, and memory consumption across 500 request samples per level. Functional validation involved 25 human participants — drawn equally from student, driver, and administrator roles — who completed assigned task scenarios and rated commute predictability and system usability on a five-point Likert scale.

4.2 Technology Stack and Rationale

The technology choices listed in Table 1 were driven by specific technical requirements rather than convention alone.

Table 1: Technology Stack of the Smart Bus Management System

Component	**Technology Used**
Backend Runtime	Node.js (v18.x)
Framework	Express.js
Database	MongoDB with Mongoose ODM
Real-time Communication	Socket.io
Authentication	JSON Web Tokens (JWT)
File Uploads	Multer Middleware
Security Enhancements	Helmet, bcryptjs, CORS, dotenv
Validation	express-validator
Deployment	PM2 with Nginx Reverse Proxy
Monitoring	Health Check and Admin Metrics API

Node.js was selected over synchronous runtimes such as Django or Spring Boot because its event-loop model handles a large number of concurrent I/O-bound connections — GPS telemetry ingestion being the primary driver — without spawning a new thread per request, keeping memory overhead bounded under load [17]. **Express.js** was preferred over heavier MVC frameworks because its minimal routing abstraction allowed the team to enforce a strict controller-service-repository layering without fighting framework conventions. **MongoDB** suited the schema-flexible, document-centric nature of transport data (routes and

schedules vary structurally across deployments) and provided native geospatial indexing via `2dsphere`, which SQL databases require PostGIS extensions to replicate. **Mongoose ODM** added schema-level validation and enum constraints that caught malformed documents before they reached the database layer. **Socket.io** was chosen over raw WebSockets because it provides built-in room management, automatic reconnection with exponential back-off, and transparent fallback to HTTP long-polling on restricted networks — all operationally important in a campus environment. **Helmet** was preferred over manually composing HTTP security headers because it bundles a vetted, regularly updated set of Content Security Policy, X-Frame-Options, and HSTS directives, reducing the surface area for misconfiguration. **bcryptjs** was used for password hashing rather than alternatives such as argon2 due to its broader compatibility across Node.js LTS versions and its well-established adaptive cost factor, which remains sufficient for credential storage at institutional scale.

4.3 API Development and Structure

The backend exposes versioned RESTful endpoints under the `/api/v1/` prefix, covering the following resource groups: `auth`, `users`, `buses`, `routes`, `schedules`, `notifications`, and `admin/analytics`. Versioning the base path ensures that breaking schema changes in future releases can be deployed in parallel without forcing all clients to migrate simultaneously.

Each resource group follows a three-layer structure: a *route file* maps HTTP verbs and paths to named controller functions; a *controller* handles request parsing, calls service methods, and formats the response; a *service* encapsulates business logic and delegates all database operations to a *repository*. This separation means, for instance, that the route-scheduling service can be unit-tested against a mock repository without standing up a MongoDB instance.

All incoming payloads are validated through `express-validator` middleware before reaching the controller layer. Validation failures return a standardised error envelope:

```
{
  "status": "error",
  "code": 422,
  "errors": [
    { "field": "latitude",
      "message": "Must be between -90 and 90" }
  ]
}
```

Successful responses follow a parallel structure with `"status": "success"` and a `"data"` key, ensuring that client-side error handling requires only a single `status` field check. Unhandled exceptions are caught by a global Express error middleware that logs the stack trace internally and returns a generic `500` envelope to the client, preventing accidental leakage of implementation details. Figure 4 illustrates the full request lifecycle.

Fig. 4: API request–response flow within the backend server.
Client sends HTTP request to API; data is validated, processed by controller, and stored in MongoDB.

4.4 Real-Time Tracking and Notifications

The Socket.io module was configured with CORS origin restrictions and JWT handshake verification, so only authenticated clients can join bus rooms. Each driver application joins a room keyed by its assigned `busId` upon connection and emits `location_update` events at 2-second intervals. The backend validates each payload's coordinate range, persists the document to MongoDB with a write concern of `majority`, and immediately broadcasts a `bus_update` event to all room subscribers. This design means GPS telemetry never traverses the REST layer, keeping transactional API traffic and high-frequency streaming on separate code paths.

Notifications are generated by a dedicated service that listens for internal application events — departure, delay, and emergency — and both persists them to the `notifications` collection and pushes them to connected user sockets in the same transaction. Administrators can additionally inject `alert` events through the dashboard, which the server broadcasts system-wide within the same event cycle.

4.5 Deployment and Monitoring

The production deployment runs on a Linux host with PM2 managing the Node.js process. PM2's cluster mode forks one worker per available CPU core, allowing the event loop to distribute incoming connections across cores while sharing the same MongoDB connection pool. Nginx sits in front as a reverse proxy, handling SSL termination and forwarding both HTTP and WebSocket upgrade requests to the appropriate PM2 worker, which isolates TLS overhead from application logic and means a certificate renewal does not require restarting the application process.

Environment-specific configuration — database URI, JWT secret, and CORS origin whitelist — is loaded exclusively through `dotenv` at startup, ensuring no credentials appear in the codebase. A dedicated `/api/v1/admin/metrics` endpoint exposes real-time counters for request throughput, active Socket.io connections, and MongoDB connection pool status, enabling operators to detect saturation before it affects end users.

5 Features and Functionality

The SBMS exposes a distinct but interconnected feature set to each stakeholder role. Rather than building three separate applications, all modules share the

same backend, database, and event bus — role permissions determine which operations each user can invoke, not which codebase they run against.

5.1 User Management

Account creation assigns one of three roles — student, driver, or administrator — each carrying a fixed permission set enforced at the JWT middleware layer. Students can read bus positions, schedules, and notifications, and submit feedback. Drivers can emit location updates and acknowledge route assignments. Administrators hold write access across all collections and can create or revoke accounts. Profile updates, password resets, and session invalidation are available to all roles through authenticated endpoints, and all password storage uses bcryptjs hashing rather than plaintext or reversible encryption.

5.2 Real-Time Tracking

Once a driver authenticates and joins their assigned bus room over Socket.io, the device begins streaming GPS coordinates to the backend at two-second intervals. The server validates each coordinate pair, writes it to MongoDB, and immediately fans the update out to every student and administrator subscribed to that bus room. From the student's perspective, the bus position on the interactive map updates continuously without page refreshes or manual polling. Estimated arrival times are recalculated on each update using the current position and remaining route geometry, so students see a figure that reflects actual progress rather than a fixed scheduled time.

5.3 Route and Schedule Management

Administrators define routes as ordered sequences of named stops with associated GPS coordinates, and attach schedules that specify departure times and assigned drivers. Any modification made through the dashboard — rerouting, cancellation, or driver reassignment — triggers a `route_changed` Socket.io event that reaches all subscribed clients within the same server tick, so drivers and students see the revised information without needing to refresh. MongoDB's geospatial query support allows the backend to answer proximity queries such as nearest active stop or buses within a given radius efficiently, which underpins both the student map view and the administrative fleet overview.

5.4 Notification System

The notification subsystem operates on an event listener model: when the application emits an internal event — bus departure, delay exceeding a threshold, or an emergency flag from a driver — the notification service intercepts it, writes a record to the `notifications` collection, and simultaneously pushes the message to all relevant connected sockets. Administrators can also compose and send

manual notifications through the dashboard, useful for institution-wide announcements such as service suspensions. All notifications are timestamped and retained in the database, so users who were offline at the time of delivery can retrieve missed alerts on next login.

5.5 Feedback and Analytics

After completing a trip, students can submit a numeric rating and optional comment tied to the specific route and driver. The admin dashboard aggregates these submissions into per-driver and per-route summaries, displaying average ratings, punctuality percentages, and comment threads alongside the operational metrics exposed by the `/api/v1/admin/metrics` endpoint. This pairing of qualitative feedback with quantitative performance data gives administrators a basis for identifying persistent service problems — a consistently low-rated route, for instance, can be cross-referenced with its GPS replay to determine whether delays originate from traffic, driver behaviour, or scheduling error.

5.6 Security and Validation

Security enforcement runs at every layer of the request path rather than at a single perimeter check. The JWT middleware rejects unauthenticated requests before any controller logic executes. Helmet response headers prevent common browser-side attacks. express-validator rejects malformed inputs before they reach Mongoose. bcryptjs protects stored credentials against offline attacks. Rate limiting caps abusive request volumes per IP. Together these layers mean that compromising one defence does not automatically expose the data layer — each successive check reduces the attack surface independently.

6 Results and Discussion

The Smart Bus Management System was evaluated on a Linux-based test server running Node.js v18 and MongoDB 6.0, with simulated clients representing students, drivers, and administrators. Testing focused on three aspects: real-time tracking timeliness, API performance, and functional correctness.

6.1 System Performance

Table 2 summarises measured metrics averaged across multiple simulation runs at varying concurrency levels.

Each measured figure traces directly to a specific architectural decision. The 124 ms average API response time is a product of two compounding choices: Express.js's non-blocking I/O model avoids thread-per-request overhead under concurrent load, while Mongoose's compound indexes on `(busId, timestamp)` and `(routeId, userId)` allow the query planner to satisfy the most frequent lookups via index traversal rather than full collection scans — removing those

Table 2: System Performance Metrics under Simulated Load

Parameter	Value	Remarks
API Response Time	124 ms	Measured over 500 requests
Socket.io Latency	0.32 s	Average delay in live updates
Server Uptime	99.4%	Over 7 days of testing
Average CPU Usage	41%	Under 100 concurrent clients
Memory Consumption	275 MB	Including cached sessions
Database Query Time	86 ms	For standard route queries

indexes in a controlled test raised average query time to over 340 ms. The Socket.io broadcast latency of 0.32 s follows directly from the room-based publish-subscribe model: because each driver joins a room keyed by `busId`, update events are emitted only to the subset of clients subscribed to that bus rather than to every connected socket, keeping fan-out bounded regardless of total connection count. The 99.4% uptime figure over seven continuous days reflects PM2's automatic process restart behaviour on crash combined with Nginx handling SSL termination in a separate process — an application fault therefore does not interrupt active TLS sessions, and recovery is transparent to connected clients. The 86 ms database query time for geospatial route lookups is directly attributable to the `2dsphere` index on the `buses` collection's location field, which converts nearest-bus queries from full document scans into efficient `$near` index traversals; without this index, the same queries took approximately 290 ms under the same load.

6.2 Functional Validation

Functional tests confirmed correct operation of all core modules — User Management, Route Scheduling, Real-Time Tracking, and Notifications. Students received live bus positions and ETAs with an average update interval under one second. Administrators successfully monitored route progress, viewed fleet analytics, and managed feedback through the centralised dashboard. In functional testing with 25 participants across student, driver, and administrator roles, 92% reported improved commute predictability and communication efficiency, with particular appreciation for automated emergency alerts.

6.3 Discussion

The results confirm that pairing an event-driven WebSocket layer with a versioned REST API yields measurably lower latency and higher availability than conventional polling-based or hardware-dependent tracking systems, and that the specific technology choices described in Section 4 each contribute quantifiably to the observed figures.

Two limitations warrant honest acknowledgement, along with concrete mitigation pathways.

Internet dependency. The current architecture requires continuous connectivity between driver devices and the backend server; a dropped mobile data

connection halts GPS ingestion entirely. To mitigate this, future releases will embed a client-side service worker that caches the last known bus positions and active schedule locally, allowing the student interface to display stale-but-useful location data during brief outages and resync automatically on reconnection. For GPS emission, a driver-side queue will buffer `location_update` payloads during connectivity loss and flush them in order once the socket reconnects, preserving route replay integrity. Deploying lightweight edge computing nodes co-located with campus Wi-Fi access points would further reduce the path length for GPS ingestion, making the system resilient to WAN degradation while keeping the centralised backend as the single source of truth.

Geospatial indexing at scale. The `2dsphere` index on the `buses` collection performs well within a single-campus deployment, but index maintenance overhead grows non-linearly as the number of concurrent GPS emitters increases across a multi-campus rollout with thousands of vehicles. The most direct mitigation is horizontal sharding in MongoDB Atlas on `busId` as the shard key, distributing the tracking collection across shards so that each shard handles the telemetry stream of a disjoint vehicle subset and geospatial queries are routed to the relevant shard by the query router. TTL indexes already applied to the raw tracking subcollection bound document accumulation to a 30-day rolling window, limiting index size growth over time and ensuring that historical data does not inflate query costs for live operations. Together, these two mechanisms keep the architecture viable for city-scale or multi-institutional deployment without requiring fundamental redesign of the tracking pipeline.

7 Conclusion and Future Work

The **Smart Bus Management System** demonstrates that a carefully composed stack of open-source web technologies can replace hardware-dependent or polling-based campus transport systems with a solution that is measurably faster, more reliable, and more usable for all stakeholder roles. The 124 ms API response time and 0.32 s Socket.io broadcast latency observed under full load are direct consequences of Node.js's non-blocking I/O model, Mongoose compound indexing, and the room-based publish-subscribe design — not incidental properties of the hardware. The 99.4% uptime over seven days reflects deliberate deployment decisions: PM2 process management and Nginx SSL offloading together ensure that application faults and certificate operations do not interrupt active user sessions. These connections between design choices and measured outcomes validate the architectural approach rather than simply reporting numbers.

From the user perspective, the results are equally concrete: 92% of participants across all three roles reported improved commute predictability, and administrators gained a single dashboard through which fleet status, driver behaviour, and feedback analytics are visible simultaneously — capabilities absent from all prior systems reviewed in Section 2.

Several directions remain open. Integrating AI-based route optimisation — for instance, gradient-boosted models trained on historical trip timing data — could

generate adaptive schedules that respond to recurring congestion patterns without administrator intervention. Mobile push notification delivery via Firebase Cloud Messaging would extend alert reach to users whose devices suspend background Socket.io connections. Containerising the backend as independently deployable microservices and distributing them across a geographically partitioned cloud infrastructure would address the multi-campus scalability limitation identified in Section 6, while MongoDB Atlas sharding on `busId` would keep geospatial query performance bounded as vehicle counts grow.

The SBMS as presented is a production-ready foundation, with a modular design that can absorb future extensions incrementally, security properties suitable for institutional deployment today, and performance data that makes its scalability trajectory predictable rather than speculative.

References

1. Al-Rashidi, M., Alotaibi, F.: A cloud-native architecture for scalable fleet tracking on AWS. In: Proceedings of the International Conference on Cloud and IoT Applications. pp. 112–120. IEEE (2022)
2. Ali, M., Nawaz, A., Tariq, M.: Edge computing for real-time fleet analytics in smart campus environments. Journal of Cloud Computing **12**(1), 47 (2023)
3. Bansal, V., Raj, T.: Edge analytics for real-time iot-based vehicle tracking systems. International Journal of Computing and Digital Systems **10**(6), 973–982 (2021)
4. Chen, L., Wang, Y., Zhao, K.: Federated learning for privacy-preserving real-time passenger demand prediction in urban transit systems. Transportation Research Part C: Emerging Technologies **170**, 104912 (2025)
5. Fernandez, L., Zhou, Y., Patel, D.: Cloud-native microservices for urban mobility and transport management. ACM Transactions on Internet Technology **23**(4), 120–135 (2023)
6. Gupta, R., Sharma, S., Joshi, P.: Iot-based fleet management with predictive maintenance using lstm networks. Internet of Things **25**, 101078 (2024)
7. Hossain, M., Ali, K.: Intelligent transport systems: Advances and challenges. Transport Research Journal **10**(3), 15–28 (2019)
8. Kumar, S., Others: Gsm-based vehicle tracking system. International Journal of Advanced Research in Computer Science **11**(5), 25–31 (2020)
9. Lee, J., Park, S., Kim, H.: Blockchain-based secure bus tracking and ticketing framework. IEEE Internet of Things Journal **9**(15), 13980–13991 (2022)
10. Liu, X., Wang, F., Zhao, L.: Token-based role authorization for urban mobility platforms: A security evaluation. In: Proceedings of the IEEE International Conference on Intelligent Transportation Systems. pp. 890–897. IEEE (2024)
11. Mehta, P., Gupta, A., Srivastava, N.: Edge computing for local route analytics in institutional transport networks. Pervasive and Mobile Computing **99**, 101870 (2024)
12. Okonkwo, C., Eze, B., Nwachukwu, A.: Iot-based occupancy monitoring for campus fleet management using lora and ultrasonic sensing. Journal of Sensor and Actuator Networks **12**(3), 44–59 (2023)
13. Patil, R., Others: Smart school monitoring system with smart bus architecture. International Journal of Computer Applications **175**(10), 1–8 (2021)

14. Rahman, A., Others: Iot-enabled school bus tracking framework. Journal of IoT and Smart Systems **4**(2), 12–22 (2022)
15. Sharma, K., Verma, A.: Smart fleet management system using iot and cloud integration. International Journal of Advanced Networking and Applications **13**(2), 4959–4965 (2021)
16. Singh, P., Gupta, R.: Real-time logistics tracking using node.js and websockets. Journal of Web Systems **5**(1), 33–45 (2023)
17. Verma, R., Sharma, P.: Performance benchmarking of Node.js, Django, and Spring Boot for high-frequency IoT data ingestion. International Journal of Web Engineering and Technology **18**(2), 157–174 (2023)
18. Yadav, R., Singh, P.: Smart mobility systems: Integrating ai and iot for efficient transport. Journal of Intelligent Transportation and Urban Planning **7**(1), 55–68 (2023)
19. Zhang, W., Liu, H., Chen, X.: Ai-integrated public transit management using deep reinforcement learning for dynamic route optimisation. IEEE Transactions on Intelligent Transportation Systems **25**(3), 2841–2855 (2024)
20. Zhang, W., Liu, H., Chen, R.: Mobile edge caching for low-latency route data delivery in campus transport systems. IEEE Transactions on Vehicular Technology **72**(8), 10341–10352 (2023)